

ENGINEERING REBOOT 2026

Day 22 Engineering Workbook

Input Validation & Defensive Programming

Implementation Edition v1.0

Purpose: Get you coding today. This is the compact implementation workbook, not the full publication chapter.

1. Mission

Day 22 moves from recovering from bad input to preventing bad input from reaching business logic. The engineering goal is to validate assumptions before processing data.

Engineering Principle: Never trust external input. Validate it at the boundary before using it.

2. Today's Learning Objectives

- Distinguish validation from exception handling.
- Build reusable validation loops.
- Validate numeric input, ranges, menus, blank strings, simple email format, and positive salary values.
- Separate validation from business logic.
- Produce evidence for success and failure-recovery paths.

3. Repository Targets

C:\Projects\Engineering-Reboot-2026\01-Week-05\Exercises\Day-22\

C:\Projects\Engineering-Reboot-2026\01-Week-05\Projects\Day-22\

4. Core Concepts

Term	Meaning
Validation	Checks that data satisfies expected rules before the program uses it.
Defensive Programming	Designing software under the assumption that users and external systems may provide invalid data.
Trust Boundary	Any point where data enters from outside the program, such as keyboard input, files, APIs, or configuration.
Fail Fast	Reject invalid data as soon as it is detected.
Business Rule	A rule from the problem domain, such as age must be between 18 and 65.

5. Validation Pipeline

Recommended order: input -> strip whitespace -> reject empty input -> type conversion -> range/business-rule validation -> processing.

6. Required Exercises

Exercise 1 - numeric_validation.py

Objective: Create `read_number(prompt: str) -> float`. It should loop until `float(input(prompt).strip())` succeeds. Catch `ValueError` and print a clear retry message.

Expected behavior: Enter amount: abc -> Invalid numeric input. Please try again. Then Enter amount: 145.75 -> Accepted value: 145.75

Exercise 2 - range_validation.py

Objective: Create `read_age(prompt: str) -> int`. It should accept only whole numbers between 18 and 65. Reject non-integers and out-of-range values.

Expected behavior: abc -> Please enter a whole number. 17 -> Age must be between 18 and 65. 53 -> Accepted age: 53

Exercise 3 - menu_validation.py

Objective: Create `VALID_CHOICES = {"1", "2", "3", "4"}`. Create `read_menu_choice() -> str` that loops until the input is one of those choices.

Expected behavior: 7 -> Invalid menu selection. abc -> Invalid menu selection. 3 -> Option accepted.

```
# numeric_validation.py example pattern
def read_number(prompt: str) -> float:
    while True:
        try:
            return float(input(prompt).strip())
        except ValueError:
            print("Invalid numeric input. Please try again.")
```

7. Mini Project - Validated Registration System

Required filename: `validated_registration_system.py`

Engineering scenario: Build a console-based employee registration workflow that prevents invalid information from entering the system.

Field	Validation Rule
Employee Name	Not empty, strip whitespace, minimum length 2.
Employee Age	Whole number, 18 through 65.
Email Address	Not empty, contains @ and . (simple validation only).
Annual Salary	Numeric and greater than zero.

Recommended Functions

- `read_name()` -> str
- `read_age()` -> int
- `read_email()` -> str
- `read_salary()` -> float
- `display_registration(name: str, age: int, email: str, salary: float)` -> None
- `main()` -> None

Expected Project Output

Employee Registration

Enter employee name: Glenn Ogapong

Enter age: abc

Please enter a whole number.

Enter age: 17

Age must be between 18 and 65.

Enter age: 53

Enter email: glenn

Invalid email format.

Enter email: glenn@example.com

Enter annual salary: -100

Salary must be greater than zero.

Enter annual salary: 95000

Registration Complete

Employee Name : Glenn Ogapong

Age : 53

Email : glenn@example.com

Salary : \$95,000.00

8. Evidence Checklist

- `numeric_validation-Code.png`
- `numeric_validation-T01-Execution.png`
- `range_validation-Code.png`
- `range_validation-T01-Execution.png`
- `menu_validation-Code.png`
- `menu_validation-T01-Execution.png`
- `validated_registration_system-Code.png`
- `validated_registration_system-T01-Success.png`
- `validated_registration_system-T02-Invalid-Age.png`
- `validated_registration_system-T03-Invalid-Email.png`
- `validated_registration_system-T04-Invalid-Salary.png`
- `validated_registration_system-T05-Blank-Name.png`
- `Git-Operations.png`
- `Notes.md` updated
- `Journal.md` updated

9. Git Workflow

git status

git add .

git commit -m "Day 22 - Completed exercises and documentation"

git push

git status

10. Stretch Goals

- Validate phone number: ten digits only.
- Validate ZIP code: five digits only.
- Centralize constants: MINIMUM_AGE, MAXIMUM_AGE, MINIMUM_NAME_LENGTH.
- Print a validation summary.
- Write registration events to registration_log.txt.

11. Notes and Journal Prompts

Notes.md: Define validation, defensive programming, trust boundary, fail fast, business rule, and input sanitization. Compare validation vs exception handling.

Journal.md: Which validation rule was hardest? Which function is most reusable? What would another engineer suggest in review?

12. Engineering Takeaways

1. Validate input before processing it.
2. Defensive programming prevents predictable failures.
3. Validation and exception handling complement one another.
4. Separate validation from business logic.
5. Small focused validation functions are reusable.
6. Centralize business rules and constants.
7. Reject invalid data as early as possible.
8. Every external input crosses a trust boundary.
9. Good software protects the integrity of its own data.
10. Professional engineers design software for future maintainers.

End of Day 22 Engineering Workbook